

Le cours est inspiré du MOOC Initiation Machine Learning de Supelec

<https://openclassrooms.com/fr/courses/4011851-initiez-vous-au-machine-learning/>

- ▶ La data science augmente les capacités d'analyse classique, afin d'aider les entreprises à prendre des décisions. Elle s'appuie pour cela sur des données utiles.
- ▶ Cycle de travail:
 1. Récupérer les données
 2. Nettoyer les données
 3. Explorer les données
 4. Créer le modèle
 5. Evaluer la qualité du modèle
 6. Déployer les algorithmes

Au sein du cycle de travail du data scientist, le machine learning désigne l'ensemble des méthodes de modélisation statistique à partir des données.

- ▶ L'objectif du machine learning est de trouver un modèle qui effectue une approximation de la réalité (le phénomène à l'origine des données).

Ce modèle est utilisé pour pouvoir effectuer des prédictions.

Les modèles paramétriques sont représentés par un ensemble de paramètres qu'on mettra dans un vecteur.

- ▶ **Fonction de risque ou d'erreur** (Fonction de "loss")

La mesure de l'éloignement des données par rapport à la prédiction obtenu du modèle

Le plus utilisée est la fonction quadratique (la distance Euclidienne)

On somme sur toutes les données, l'erreur effectuée par le modèle.

Il faut **minimiser** cette erreur

Il y a aussi maximum vraisemblance qu'on peut choisir.
Dans ce cas, il faut le maximiser.

Apprentissage Supervisé

- ▶ Données (DataSet)
- ▶ Modèles et ses paramètres
- ▶ Fonction de coût
- ▶ Algorithme d'apprentissage

Dataset

<i>Attributs (Features)</i>		<i>Cible (Target)</i>
<i>Surface m²</i>	<i>nb chambres</i>	<i>Prix KEuros</i>
120	4	300
50	2	280
17	1	200
⋮	⋮	⋮

Représentation mathématique

- ▶ On représente donnée (objet) X_i par un vecteur x_i défini dans un espace de description composé de plusieurs variables.
- ▶ Les données (n données) qui a p attributs sont définies dans une matrice, X et le vecteur y pour les cibles associés.

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

- ▶ L'ensemble des couples observés

$$\{(x_1, y_1), \dots, (x_i, y_i), \dots, (x_n, y_n)\}$$

où x_i est la i ème ligne de X .

X est l'ensemble d'entraînement ou d'apprentissage
(ou ensemble des données annotées ou étiquetées)

- ▶ **Régression** : lorsque la valeur cible est continue (une valeur réelle)
- ▶ **Classification**: (Classement, catégorisation) lorsque la valeur cible est discrète.

Classification binaire si la valeur cible a 2 valeurs

Modèle

- ▶ Le but est de trouver une fonction modélisant la relation entre X et la variable cible y :

$$f(X) = y$$

- ▶ Les données observées sont bruitées:

$$f(X) = y + \epsilon$$

où ϵ est l'erreur.

- ▶ On souhaite approximer en commettant le moins d'erreurs possibles afin de faire de bonnes prédictions pour des données qui ne sont pas encore observées.

Fonction de coût (perte)

Distance Euclidienne

$$\sum_1^n (y_i - f(x_i))^2 = \sum_1^n (\epsilon_i)^2$$

Mean Squared Erreur (MSE)

$$\frac{1}{n} \sum_1^n (y_i - f(x_i))^2 = \frac{1}{n} \sum_1^n (\epsilon_i)^2$$

Classification par la méthode des plus proches voisins (KNN)

On estime la cible pour une nouvelle donnée en regardant ces K plus proches voisins

- ▶ Déterminez le nombre de voisins K
- ▶ Prenez les K voisins selon les distances calculées
- ▶ Parmi K voisins, comptez le nombre de points appartenant à chaque catégorie
- ▶ Attribuez le nouveau point à la catégorie la plus présente parmi ses K voisins

Régression linéaire

- ▶ La régression linéaire simple consiste à prendre pour hypothèse que la relation est un polynôme de degré 1 de X

$$f(X) = y = aX + b$$

- ▶ a, b sont les paramètres du modèle et on souhaite les estimer.
 $\hat{a}$ et $\hat{b}$ sont les estimations de a, b .
- ▶ La fonction de perte en terme de a et b :

$$\sum_1^n (y_i - (ax_i + b))^2$$

- ▶ Le but est de minimiser la fonction de perte.
Algorithme (**la droite des moindres carrés**), ces paramètres sont calculés comme suit

$$\hat{b} = \bar{y} - \hat{a}\bar{x}$$
$$\hat{a} = \frac{\sum_1^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_1^n (x_i - \bar{x})^2}$$

où $\bar{y} = \frac{1}{n} \sum_1^n y_i$ et $\bar{x} = \frac{1}{n} \sum_1^n x_i$

- ▶ La fonction de prédiction:

$$\hat{f} = \hat{b} + \hat{a}x$$

Le modèle de prédiction pour une nouvelle donnée x_{new} :

$$\hat{y} = \hat{b} + \hat{a}x_{new}$$

Descente de Gradient Stochastique

Tutoriel très bien fait:

<https://machinelearnia.com/descente-de-gradient/>

- La fonction de perte atteint un minimum lorsque les dérivées partielles par rapport à a et b valent 0.

Pour simplifier, on considère la fonction de perte suivante:

$$J(a, b) = \frac{1}{n} \sum_1^n (y_i - f(x_i))^2 = \frac{1}{n} \sum_1^n (\epsilon_i)^2$$



$$\frac{\partial J(a, b)}{\partial b} = \frac{-2}{n} \sum_{i=1}^n (y_i - (ax_i + b))$$

$$\frac{\partial J(a, b)}{\partial a} = \frac{-2}{n} \sum_{i=1}^n (y_i - (ax_i + b))x_i$$

Ces équations ont une solution unique car la fonction de perte est convexe.

On trouve les paramètres $\hat{a}$ et $\hat{b}$ par un algorithme itératif.

- ▶ Choisir les valeurs initiales pour a, b et la vitesse d'apprentissage α
- ▶ Répéter jusqu'à la convergence
 - ▶ Mélanger aléatoirement les données $x_i, 1 \dots n$
 - ▶ Pour $i = 1 \dots n$ faire

$$a = a - \alpha \frac{\partial J(a, b)}{\partial a}; \quad b = b - \alpha \frac{\partial J(a, b)}{\partial b}$$

- ▶ A chaque itération, a, b sont mis à jour en soustrayant leur propre valeur à la valeur de la pente ($\frac{\partial J(a, b)}{\partial \cdot}$) multiplié par le taux d'apprentissage (α)

Si α est trop lente, le modèle peut mettre longtemps, mais si la vitesse est trop grande, le modèle ne jamais converger.